

Chapter 8

Simulation of Queues and Related Models

Lectures on Monte Carlo Theory

Leskelä & Vihola

Chapter Overview

- 1 8.1 Preliminaries
- 2 8.2 Modeling Arrival Streams
 - 8.2.1 Homogeneous Poisson Process
 - 8.2.2 Non-homogeneous Poisson Process
 - 8.2.3 Poisson Process on General Spaces
- 3 8.3 Birth and Death Processes / Markovian Queues
- 4 8.4 Basic Concepts of Queueing Theory
- 5 8.5 Discrete-Event Simulations
 - 8.5.1 GI/G/c Queue
- 6 8.6 Slotted ALOHA
- 7 8.7 Ergodically Stable Processes
 - 8.7.1 Asymptotic Variance Formula
 - 8.7.2 TAVC for Other Systems
- 8 8.8 Regenerative Processes
- 9 8.9 Error Analysis of M/M/1 and M/G/ ∞
- 10 8.10 Bibliographical Notes

8.1 Preliminaries – Queueing Systems

A “system” receives tasks and processes them:

- Tasks (jobs, customers, calls) arrive according to some mechanism.
- Tasks are processed in some manner and eventually leave.

Two representations of system characteristics:

- *Discrete-time sequence*: e.g. waiting time W_i of the i -th arriving task.
- *Continuous-time process*: e.g. number of tasks $L(t)$ in the system at time t .

Time Averages vs. Ensemble Averages

Time Average

For a process $X(t)$, the *time average* over $[0, t]$ is:

$$\hat{X}(t) = \frac{1}{t} \int_0^t X(s) ds.$$

Represents the average value observed in a single long simulation run.

Ensemble Average

The *ensemble average* at time t is $\mathbb{E}[X(t)]$, the expected value over all possible realizations.

Key question: When do $\hat{X}(t) \rightarrow \mathbb{E}[X^*]$ as $t \rightarrow \infty$?

- X^* is a random variable with the *stationary distribution*.
- Answer: when the process is *ergodically stable* (Section 8.7).

Steady-State Characteristics

Steady-State Mean and Variance

For a *continuous-time* process $X(t)$ with stationary distribution:

$$\mathbb{E}[X^*] = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t X(s) ds, \quad \mathbb{V}\text{ar}[X^*] = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t (X(s) - \mathbb{E}[X^*])^2 ds.$$

For *discrete-time* chains (X_k) the analogues are:

$$\mu = \mathbb{E}[X^*] = \lim_{k \rightarrow \infty} \frac{1}{k} \sum_{j=1}^k X_j, \quad \sigma^2 = \mathbb{V}\text{ar}[X^*] = \lim_{k \rightarrow \infty} \frac{1}{k} \sum_{j=1}^k X_j^2 - \mu^2.$$

Biased estimators from a single run:

$$\hat{\mu}(k) = \frac{1}{k} \sum_{j=1}^k X_j, \quad \hat{\sigma}^2(k) = \frac{1}{k} \sum_{j=1}^k (X_j - \hat{\mu}(k))^2.$$

8.2 Modeling Arrival Streams

Counting process $N(t)$:

- $N(t)$ = number of arrivals up to time t ($t \geq 0$).
- Piecewise constant, non-decreasing, unit jumps; $N(0) = 0$.
- Right-continuous in t .

Arrival times (A_n) and inter-arrival times (τ_i):

Inter-Arrival Times and Counting Process

$$\tau_1 = A_1, \quad \tau_i = A_i - A_{i-1}, \quad (i = 2, \dots) \quad (8.2.1)$$

$$N(t) = \#\{i \geq 1 : A_i \leq t\}. \quad (8.2.2)$$

τ_i = inter-arrival times; A_n = arrival instants.

Interval Counts

For $s < t$: $N(s, t] := N(t) - N(s) = \#\{i \geq 1 : s < A_i \leq t\}$.

8.2.1 Homogeneous Poisson Process

Definition 8.2.1

A counting process $N(t)$ is a **homogeneous Poisson process** with intensity $\lambda > 0$ if the inter-arrival times (τ_j) are i.i.d. $\text{Exp}(\lambda)$.

Proposition 8.2.2 (Equivalent Characterization)

$N(t)$ is a Poisson process with intensity λ iff:

- 1 Numbers of points in disjoint intervals are *independent*.
- 2 $N(t, t + h] \sim \text{Poi}(\lambda h)$ for all $t, h > 0$.

Key facts:

- $N(t) \sim \text{Poiss}(\lambda t)$.
- $\mathbb{P}(N(t, t + h] = 1) = \lambda h e^{-\lambda h} = \lambda h + o(h)$.
- λ = average arrival rate per unit time.
- Arrival times: $A_i = \tau_1 + \cdots + \tau_i$.

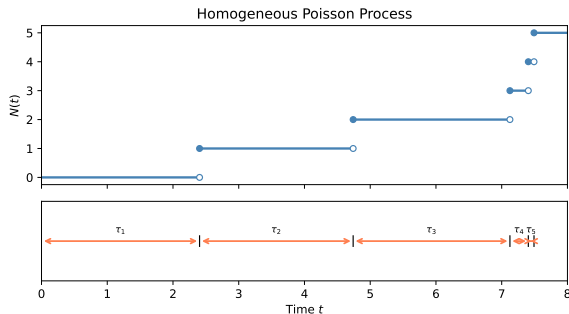
Simulation of Homogeneous Poisson Process

Algorithm PoiPro(λ)

- 1 Create empty list A
- 2 Generate $U \sim \mathcal{U}(0, 1)$, set $t = -\log(U)/\lambda$
- 3 **while** $t \leq t_{\max}$ **do**
- 4 $A.append(t)$
- 5 Generate $U \sim \mathcal{U}(0, 1)$, $\tau = -\log(U)/\lambda$
- 6 $t = t + \tau$
- 7 **end while**
- 8 **return** A

Alternative: PoiProSorted(λ)

- 1 Sample $K \sim \text{Poi}(\lambda t_{\max})$
- 2 Generate $A_1, \dots, A_K$ i.i.d. $\mathcal{U}(0, t_{\max})$
- 3 Return sorted values $(A_{(1)}, \dots, A_{(K)})$



Uniform order statistics: Given $N(t) = n$, the arrival times are the ordered statistics of n i.i.d. $\mathcal{U}(0, t)$ variables.

Python: Homogeneous Poisson Process

```
import numpy as np

def simulate_poisson(lam, t_max, rng=None):
    """Algorithm PoiPro: simulate Poisson(lambda) on [0, t_max]."""
    if rng is None:
        rng = np.random.default_rng()
    arrivals = []
    t = -np.log(rng.uniform()) / lam          # first inter-arrival
    while t <= t_max:
        arrivals.append(t)
        t += -np.log(rng.uniform()) / lam     # next inter-arrival
    return np.array(arrivals)

# Example: lambda=2, simulate on [0,10]
rng = np.random.default_rng(42)
A = simulate_poisson(lam=2.0, t_max=10.0, rng=rng)
print(f"Number of arrivals: {len(A)}")      # Expected: ~20
print(f"Arrival times: {A[:5].round(3)}")
```

8.2.2 Non-homogeneous Poisson Process

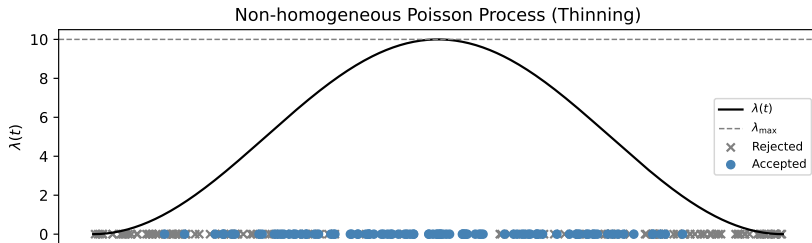
Definition 8.2.3

$N(t)$ is a **non-homogeneous Poisson process** with intensity $\lambda(t)$ if:

- 1 Numbers of points in disjoint intervals are independent.
- 2 $N(t, t + h]$ has Poisson distribution with mean $\int_t^{t+h} \lambda(s) ds$.

Thinning Method (Proposition 8.2.4)

Let $N_h(t)$ be homogeneous Poisson with intensity $\lambda_{\max} \geq \lambda(t)$. Retain each point A_i independently with probability $p(t) = \lambda(t)/\lambda_{\max}$. The thinned process is non-homogeneous Poisson with intensity $\lambda(t)$.



Python: NHPP Thinning (Algorithm 75)

```
import numpy as np

def nhpp_thinning(lam_func, lam_max, t_max, rng=None):
    """NH-Poi-thinning: simulate NHPP with intensity lam_func(t)."""
    if rng is None:
        rng = np.random.default_rng()
    A = []
    t = -np.log(rng.uniform()) / lam_max      # Exp(lam_max) jump
    while t <= t_max:
        p = lam_func(t) / lam_max             # acceptance probability
        if rng.uniform() <= p:
            A.append(t)
            t += -np.log(rng.uniform()) / lam_max
    return np.array(A)

# Example: daily arrival pattern lambda(t) = 10*(0.5+0.5*sin(2*pi*t/24))
lam_func = lambda t: 10*(0.5 + 0.5*np.sin(2*np.pi*t/24 - np.pi/2))
rng = np.random.default_rng(7)
A = nhpp_thinning(lam_func, lam_max=10.0, t_max=24.0, rng=rng)
print(f"Arrivals in [0,24]: {len(A)}")
```

8.2.3 Poisson Process on General Spaces

Definition 8.2.6 – Poisson Point Process on $E \subset \mathbb{R}^d$

A point process N on E with intensity measure $\Lambda(\cdot)$ is a **Poisson point process** if:

- 1 For all disjoint $B_1, \dots, B_n \in \mathcal{A}$, the counts $N(B_1), \dots, N(B_n)$ are independent.
- 2 $N(B) \sim \text{Poi}(\Lambda(B))$ for all $B \in \mathcal{A}$.

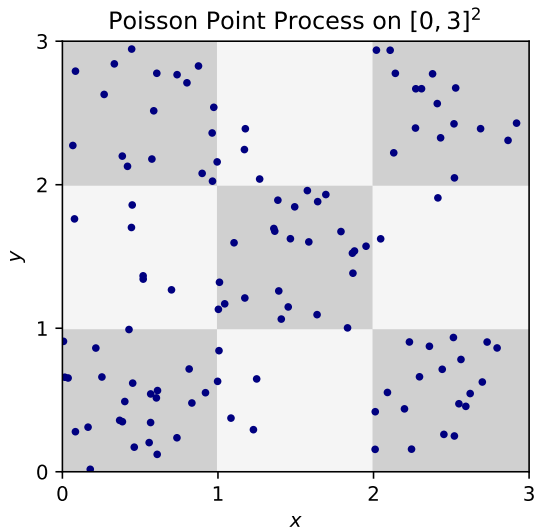
Special cases:

- Homogeneous on $[0, t_{\max}]$: $\Lambda(dt) = \lambda dt$.
- Non-homogeneous on $[0, t_{\max}]$: $\Lambda(dt) = \lambda(t) dt$.
- General $E \subset \mathbb{R}^d$: $\Lambda(d\mathbf{x}) = \lambda d\mathbf{x}$.

Algorithm 77: Finite Intensity $\Lambda(E) < \infty$

- 1 Generate $T \sim \text{Poi}(\Lambda(E))$.
- 2 Generate $\xi_1, \dots, \xi_T$ independently from E with distribution $\Lambda(d\mathbf{x})/\Lambda(E)$.
- 3 Return points $\xi_1, \dots, \xi_T$.

Poisson Point Process on $[0, 3]^2$



Example: Heterogeneous intensity on $[0, 3]^2$:

- Grey cells: intensity = 20
- White cells: intensity = 5

Simulation procedure:

- 1 Partition $E = E_1 \cup E_2 \cup \dots$
- 2 On each E_j : simulate $N_j \sim \text{Poi}(\Lambda(E_j))$ points
- 3 Points uniform within each E_j

Key property: If N_1, N_2 are independent Poisson processes with intensities Λ_1, Λ_2 , then $N_1 + N_2$ is Poisson with intensity $\Lambda_1 + \Lambda_2$.

8.3 Birth and Death Processes and Markovian Queues

Birth-and-Death Process

A continuous-time Markov process $X(t)$ with state space $\{0, 1, 2, \dots\}$:

- From state i : *birth* (jump to $i + 1$) at rate λ_i .
- From state i : *death* (jump to $i - 1$) at rate μ_i (with $\mu_0 = 0$).

Next event time: $\tau \sim \text{Exp}(\lambda_i + \mu_i)$.

Birth/death selected with probabilities $\lambda_i/(\lambda_i + \mu_i)$ and $\mu_i/(\lambda_i + \mu_i)$.

Kendall Notation for Queues: A/S/c

- A = inter-arrival distribution (M=exponential, G=general, GI=i.i.d.)
- S = service time distribution
- c = number of servers ($c = \infty$ = infinite servers)

M/M/1 Queue

M/M/1: Poisson arrivals (rate λ), exponential service (rate μ), 1 server.

Steady-State Mean Queue Length

Traffic intensity $\rho = \lambda/\mu < 1$ (stability condition):

$$\bar{l} = \mathbb{E}[L^*] = \frac{\lambda}{\mu - \lambda} = \frac{\rho}{1 - \rho}.$$

Stationary distribution: $\pi_i = (1 - \rho)\rho^i$, $i = 0, 1, \dots$

Estimation from simulation:

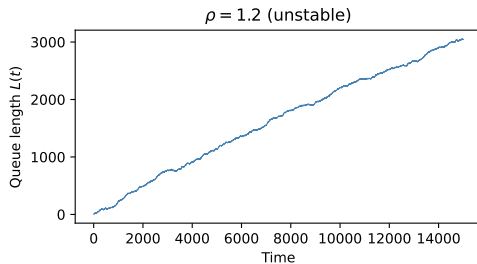
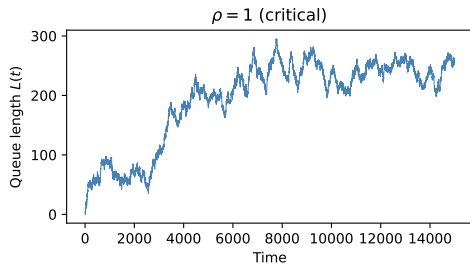
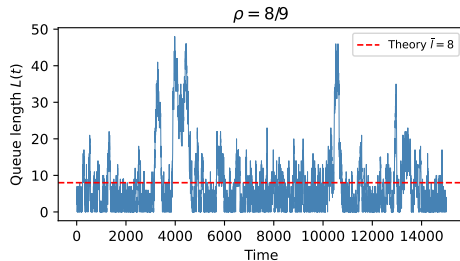
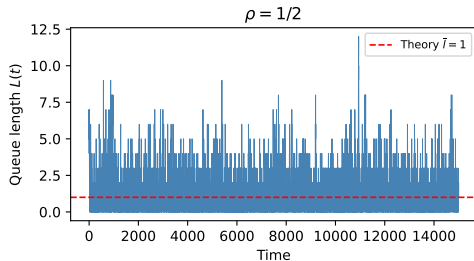
$$\hat{l}(t_{\max}) = \frac{\int_0^{t_{\max}} L(s) ds}{t_{\max}} = \frac{1}{t_{\max}} \sum_{j=0}^k y_j (a_{j+1} - a_j)$$

where a_j are event times and y_j the state between events.

Stability: The system is stable iff $\rho < 1$. For $\rho \geq 1$, $L(t) \rightarrow \infty$.

M/M/1 Simulations

M/M/1 Queue Simulations ($\mu = 1$)



Python: Birth-and-Death / M/M/1 Simulation

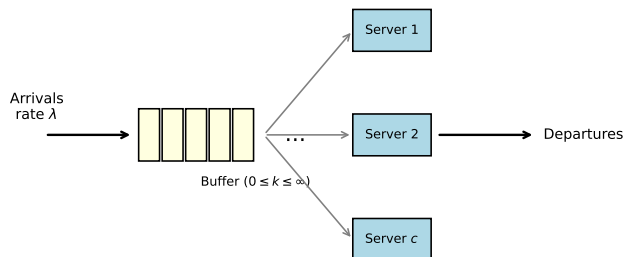
```
import numpy as np

def simulate_mm1(lam, mu, t_max, L0=0, rng=None):
    """Simulate M/M/1 queue via birth-and-death process."""
    if rng is None:
        rng = np.random.default_rng()
    t, L = 0.0, L0
    jump_times, states = [0.0], [L0]
    while t < t_max:
        rate = lam + (mu if L > 0 else 0)
        t += rng.exponential(1/rate)
        if t >= t_max:
            break
        if rng.uniform() < lam / rate:
            L += 1          # arrival (birth)
        else:
            L = max(L-1, 0) # departure (death)
        jump_times.append(t); states.append(L)
    # Compute time-averaged queue length
    ts = np.array(jump_times + [t_max])
    ss = np.array(states + [states[-1]])
    l_hat = np.sum(ss[:-1] * np.diff(ts)) / t_max
    return l_hat, jump_times, states

rng = np.random.default_rng(0)
l_hat, _, _ = simulate_mm1(0.5, 1.0, 1e4, rng=rng)
print(f"Estimated mean: {l_hat:.4f}, Theory: {0.5/0.5:.1f}")
```

8.4 Basic Concepts of Queueing Theory

GI/G/c Queueing System



System characteristics:

- A_i = arrival time of i -th task
- S_i = service time (size) of i -th task
- W_i = waiting time (before service)
- $D_i = A_i + W_i + S_i$ = departure time
- $L(t)$ = tasks in system at time t

Stability:

Traffic intensity $\rho = \mathbb{E}[S]/\mathbb{E}[T]$ where T = inter-arrival time.

Stable iff $\rho < c$ (for c servers).

Little's Law

Little's Law ($L = \lambda W$)

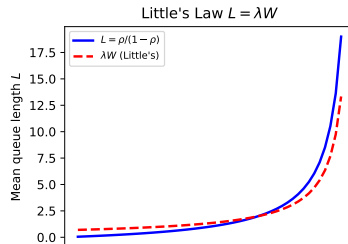
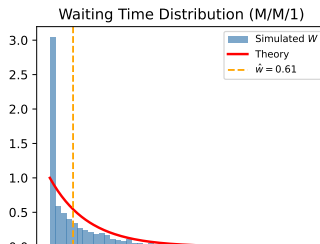
Under mild conditions, for any stable queueing system:

$$\bar{L} = \lambda \bar{W},$$

where $\bar{L}$ = steady-state mean number of tasks in system, λ = arrival rate, $\bar{W}$ = steady-state mean sojourn time.

Application to M/M/1:

- $\bar{L} = \rho/(1 - \rho)$, λ known, so $\bar{W} = \bar{L}/\lambda = 1/(\mu - \lambda)$.



GI/G/1 Waiting Time Recursion

Lindley Recursion (Eq. 8.4.1)

Under FCFS discipline, $W_1 = 0$ and:

$$W_{i+1} = (W_i + S_i - \tau_{i+1})_+.$$

W_i = waiting time of i -th task; S_i = service time; τ_{i+1} = next inter-arrival.

Stability: If $\mathbb{E}[S_1] < \mathbb{E}[\tau_1]$, then $W_i \rightarrow W_\infty$ (steady state).

Estimate:

$$\hat{w} = \frac{1}{k} \sum_{i=1}^k W_i.$$

Pollaczek-Khinchin Formula (M/G/1)

When inter-arrivals are $\text{Exp}(\lambda)$:

$$\tilde{w} = \frac{\lambda \mathbb{E}[S_1^2]}{2(1 - \lambda \mathbb{E}[S_1])}.$$

M/G/ ∞ Systems

Infinite-server system: every task is immediately served; no waiting.

M/G/ ∞ Steady-State Distribution

Arrivals: Poisson(λ); service times i.i.d. $\sim G$; $\mathbb{E}S_1 = \mu^{-1} < \infty$. With $\rho = \lambda/\mu = \lambda\mathbb{E}S$:

$$\mathbb{P}(L^* = k) = \frac{\rho^k}{k!} e^{-\rho}, \quad k = 0, 1, \dots$$

$$\mathbb{E}L^* = \rho = \frac{\lambda}{\mu}, \quad (\text{Poisson distribution})$$

Key: The steady-state distribution depends only on $\rho = \lambda/\mu$, not on the specific form of G .

Virtual waiting time (workload):

$$V(t) = \sum_{i=1}^{\infty} (W_i + S_i - t)_+ \mathbf{1}(A_i \leq t < A_{i+1}).$$

8.5 Discrete-Event Simulations

Key idea: System state changes *only at event times* (arrivals, departures). Between events, the state is constant.

Core components:

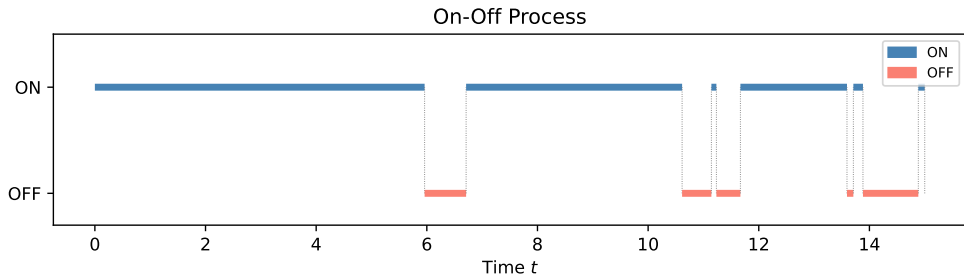
- ① **Simulation clock** t : current simulation time.
- ② **Event list** (EL): list of pending (time, event_type) pairs.
- ③ **State variables**: e.g. n = number of tasks in system.
- ④ **Output list** JS: records $(t(i), n(i))$ pairs at each event.

On-Off Process as simplest example:

- State alternates between 0 (off) and 1 (on).
- On-times $\sim F_{\text{on}}$, off-times $\sim F_{\text{off}}$.
- Steady-state probability of being on:

$$p = \frac{\mathbb{E} T^{\text{on}}}{\mathbb{E} T^{\text{on}} + \mathbb{E} T^{\text{off}}}.$$

On-Off Process



Estimator of p

$$\hat{p}(t_{\max}) = \frac{1}{t_{\max}} \int_0^{t_{\max}} K(s) ds,$$

where $K(t) \in \{0, 1\}$ is the current state. This estimator is strongly consistent.

Python: On-Off Discrete-Event Simulation

```
import numpy as np

def simulate_onoff(lam_on, lam_off, t_max, rng=None):
    """Discrete-event simulation of on-off process."""
    if rng is None:
        rng = np.random.default_rng()
    t = 0.0; state = 1  # start in ON state
    integral = 0.0      # accumulate time in ON state
    events = [(0.0, 1)]
    while t < t_max:
        # Time until next switch
        if state == 1:
            dt = rng.exponential(1/lam_on)
        else:
            dt = rng.exponential(1/lam_off)
        new_t = min(t + dt, t_max)
        integral += state * (new_t - t)
        t = new_t
        if t < t_max:
            state = 1 - state
            events.append((t, state))
    p_hat = integral / t_max
    return p_hat, events

# Example:  $E[T^{on}]=3$ ,  $E[T^{off}]=1$ ,  $p = 3/4$ 
rng = np.random.default_rng(5)
p_hat, _ = simulate_onoff(1/3, 1.0, t_max=10000, rng=rng)
```


GI/G/c Queue

Discrete-event simulation of a c -server queue with general distributions.

Single-Server GI/G/1: Algorithm 81

State: arrival time t_A , departure time t_D , queue size n .

- ① Initialize: $t_A \sim F$, $EL = [t_A, \emptyset]$.
- ② **While** $t_A < t_{\max}$, process events:
 - **Arrival** (EL empty): service $S \sim G$, $t_D = t_A + S$; next arrival.
 - **Both in EL**: if $t_A < t_D$ (arrival first): $n += 1$.
 - Otherwise (departure first): $n -= 1$; if $n > 0$, new service.

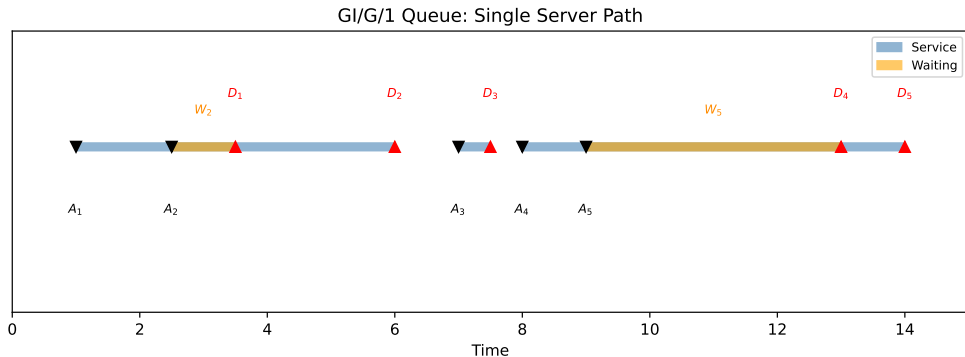
GI/G/c Workload Recursion

Vector $\mathbf{V}_i = (V_{i,1}, \dots, V_{i,c})$ of server workloads, sorted:

$$\mathbf{V}_{i+1} = \text{sort}((V_{i,1} + S_i - \tau_{i+1})_+, (V_{i,2} - \tau_{i+1})_+, \dots, (V_{i,c} - \tau_{i+1})_+).$$

Waiting time of i -th task: $W_i = V_{i,1}$ (smallest workload). Stable iff $\rho = \mathbb{E}S/\mathbb{E}\tau < c$.

GI/G/1 Queue Path



Example path: arrivals (down arrows), service intervals (blue), waiting times (orange), departures (up arrows).

Python: GI/G/1 Waiting Times

```
import numpy as np

def gig1_waiting_times(n_tasks, F_interarr, G_service, rng=None):
    """Sample waiting times W_1,...,W_n for GI/G/1 (FCFS)."""
    if rng is None:
        rng = np.random.default_rng()
    tau = F_interarr(n_tasks, rng)    # inter-arrival times
    S = G_service(n_tasks, rng)       # service times
    W = np.zeros(n_tasks)
    for i in range(1, n_tasks):
        W[i] = max(W[i-1] + S[i-1] - tau[i], 0)
    return W

# M/M/1 example: lambda=0.5, mu=1
rng = np.random.default_rng(0)
F = lambda n, r: r.exponential(2.0, n)    # Exp(1/2), mean=2
G = lambda n, r: r.exponential(1.0, n)    # Exp(1), mean=1
W = gig1_waiting_times(10000, F, G, rng)
w_hat = np.mean(W)
# Theory: w = lambda * E[S^2] / (2 * (1 - rho)) with Exp service: E[S^2]=2
rho = 0.5
w_theory = rho / (1 - rho) / 1.0    # for M/M/1: w = rho / (mu * (1 - rho))
print(f"w_hat = {w_hat:.4f}, theory = {w_theory:.4f}")
```

8.6 Slotted ALOHA and Random Access Protocols

Setup: Shared transmission channel; time divided into slots.

Packet dynamics:

- X_n = packets waiting at start of slot n .
- $A_n \sim \text{Poi}(\lambda)$ = new arrivals in slot n .
- Z_n = transmissions in slot n .
- Successful if $Z_n = 1$ (no collision).

Retransmission strategies:

- **Slotted ALOHA:** retry with fixed prob. h
- **Exp. Back-off:** $h_i = 2^{-i}$
- **Ethernet:** $h_i = 2^{-i}$ up to K attempts

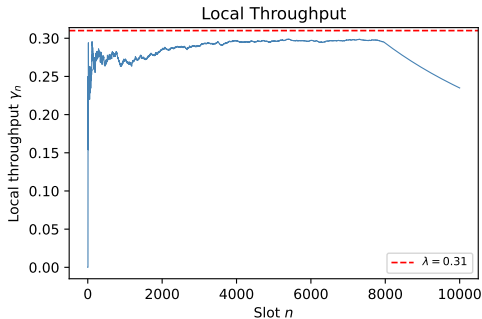
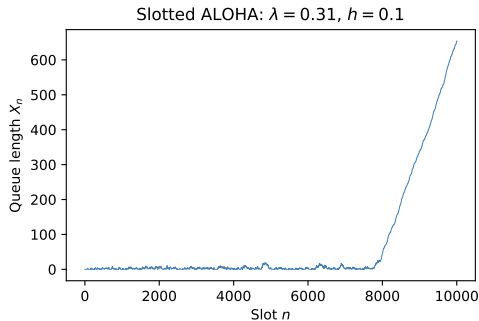
Queue Update (8.6.1)

$$X_{n+1} = X_n + A_n - \mathbf{1}(Z_n = 1), \quad n = 0, 1, \dots$$

Conditional Distribution

$$Z_n \mid X_n = k: A_n + Y_n, \quad Y_n \sim \text{Bin}(k, h)$$

Slotted ALOHA: Instability



Conditional Drift

$$\mathbb{E}[X_{n+1} - X_n \mid X_n = k] = \lambda - e^{-\lambda} k h (1 - h)^{k-1} - \lambda e^{-\lambda} (1 - h)^k.$$

As $k \rightarrow \infty$: drift $\rightarrow \lambda > 0$. **The queue grows without bound $\Rightarrow$ Slotted ALOHA is inherently unstable!**

Python: Slotted ALOHA Simulation

```
import numpy as np

def simulate_aloha(lam, h, n_slots, rng=None):
    """Simulate Slotted ALOHA with Poisson arrivals."""
    if rng is None:
        rng = np.random.default_rng()
    X = 0; Xs = [0]; successes = 0; throughputs = [0.0]
    for n in range(1, n_slots+1):
        A = rng.poisson(lam)                # new arrivals
        Y = rng.binomial(X, h) if X > 0 else 0  # retransmissions
        transmitting = A + Y
        success = int(transmitting == 1)
        successes += success
        X = max(X + A - success, 0)
        Xs.append(X)
        throughputs.append(successes / n)
    return np.array(Xs), np.array(throughputs)

# Critical region: lambda=0.31, h=0.1
rng = np.random.default_rng(11)
Xs, gammas = simulate_aloha(0.31, 0.1, 10000, rng)
print(f"Final queue: {Xs[-1]}, Final throughput: {gammas[-1]:.3f}")
```

8.7 Ergodically Stable Processes

Stationary Process

$X^*(t)$ is **stationary** if the joint distribution of $(X^*(t + h_1), \dots, X^*(t + h_d))$ does not depend on t .

- Marginal distributions: $\mathbb{E}[X^*(t)]$ and $\text{Var}[X^*(t)]$ are constant.
- Covariance function: $R(h) = \text{Cov}(X^*(t), X^*(t + h))$ depends only on h .

Ergodic Process

A stationary process $X^*(t)$ is **ergodic** if for any function g :

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t g(X^*(h_1 + s), \dots, X^*(h_d + s)) ds = \mathbb{E}[g(X^*(h_1), \dots, X^*(h_d))]. \quad (8.7.2)$$

In particular ($d = 1$):

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t X^*(s) ds = \mathbb{E}[X^*(0)].$$

Ergodically Stable Processes

Definition: Ergodically Stable

$X(t)$ is **ergodically stable** if there exists a stationary ergodic process $X^*(t)$ such that, with probability 1:

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t g(X(h_1 + s), \dots, X(h_d + s)) ds = \mathbb{E}[g(X^*(h_1), \dots, X^*(h_d))].$$

Key property: time averages converge to ensemble averages, regardless of initial state.

Practical Implication

$$\lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t X(s) ds = \mathbb{E}[X^*(0)] = I, \quad \text{a.s.}$$

So the *time average* $\hat{X}(t) = \frac{1}{t} \int_0^t X(s) ds$ is a consistent estimator of $I = \mathbb{E}[X^*]$.

Central Limit Theorem for Ergodic Processes

CLT (8.7.4)

For an ergodic process satisfying mild conditions:

$$\sqrt{t}(\hat{X}^*(t) - \mathbb{E}[X^*(0)]) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2),$$

where ς^2 is the **asymptotic variance** (TAVC).

Fundamental Error–Precision Relationship

$$b = z_{1-\alpha/2} \frac{\varsigma}{\sqrt{t}},$$

b = confidence interval half-width at level $1 - \alpha$; $z_{1-\alpha/2}$ = normal quantile.

Confidence interval:

$$\hat{X}^*(t) \pm \frac{z_{1-\alpha/2} \varsigma}{\sqrt{t}}.$$

8.7.1 Asymptotic Variance (TAVC)

Lemma 8.7.5 – TAVC Formula

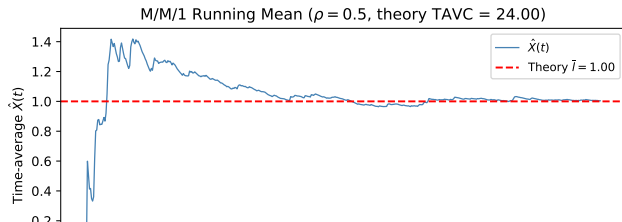
If $\int_0^\infty |R(v)| dv < \infty$ and $\int_0^\infty R(s) ds \neq 0$, then:

$$\text{Var}\left(\int_0^t X^*(s) ds\right) \sim 2t \int_0^\infty R(v) dv, \quad t \rightarrow \infty.$$

Therefore:

$$\varsigma^2 = 2 \int_0^\infty R(v) dv. \quad (8.7.9)$$

$R(h) = \text{Cov}(X^*(t), X^*(t+h)) = \text{covariance function of the stationary process.}$



TAVC for the On-Off Process

Asymptotic Variance of On-Off Process (8.7.11)

$$\varsigma^2 = \frac{(\mathbb{E} T^{\text{on}})^2 \text{Var } T^{\text{off}} + (\mathbb{E} T^{\text{off}})^2 \text{Var } T^{\text{on}}}{(\mathbb{E} T^{\text{on}} + \mathbb{E} T^{\text{off}})^3}.$$

Special case $T^{\text{on}} \sim \text{Exp}(\lambda)$, $T^{\text{off}} \sim \text{Exp}(\mu)$:

$$\varsigma^2 = 2 \frac{\lambda \mu}{(\lambda + \mu)^3} = \frac{2\rho}{\mu(1 + \rho)^3}, \quad \rho = \lambda/\mu.$$

M/M/1 Queue TAVC (8.9.3):

$$\varsigma_L^2 = \frac{2\rho(1 + \rho)}{\mu(1 - \rho)^4}.$$

This grows like $(1 - \rho)^{-4}$ as $\rho \nearrow 1$ (heavy traffic).

8.7.2 TAVC for Other Systems

Finite B&D Process (Whitt–Burman Formula, 8.7.14)

For $Y(t) = f(X(t))$ with B&D process $X(t)$, stationary mean I_f :

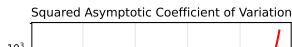
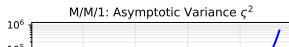
$$\varsigma_f^2 = 2 \sum_{j=0}^{n-1} \frac{1}{\lambda_j \pi_j} \left[\sum_{i=0}^j (f(i) - I_f) \pi_i \right]^2.$$

Stationary distribution: $\pi_i = \pi_0 \frac{\lambda_0 \cdots \lambda_{i-1}}{\mu_1 \cdots \mu_i}.$

M/M/n Queue TAVC

With $\rho = \lambda/\mu$, $E_n(\rho) = \sum_{j=0}^n \rho^j / j!$:

$$\varsigma^2 = \frac{2}{\lambda} \sum_{j=0}^n \frac{1}{\rho^j / j!} \cdot \frac{(\rho(E_{n-1}(\rho) - E_{j-1}(\rho)))^2}{E_n(\rho)}.$$



8.8 Regenerative Processes

Definition 8.8.1

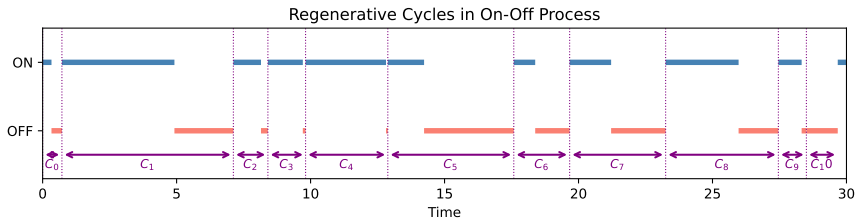
$X(t)$ is **regenerative** if there exist random times $0 =: T_0 \leq T_1 < T_2 < \dots$ such that:

- Pairs (Z_i, C_i) are independent for $i \geq 0$ and i.i.d. for $i \geq 1$, where $C_i = T_{i+1} - T_i$ (cycle length) and $Z_i = \int_{T_i}^{T_{i+1}} X(s) ds$ (integral over cycle).

The process *restarts* probabilistically at each T_i .

Examples:

- **On-off process:** regenerates when entering state 1 (on).
- **M/M/1 queue:** regenerates at times when $L(t-) = 0$ and a new arrival occurs.
- **GI/G/1 queue:** regenerates at start of busy periods.



Renewal-Reward Theorem

Proposition 8.8.2 (Renewal Reward)

If $\mathbb{E}C_1 < \infty$ and $\mathbb{E}|Z_1| < \infty$, then with probability 1:

$$\hat{X}(t) \rightarrow I = \frac{\mathbb{E} \int_{T_1}^{T_2} X(s) ds}{\mathbb{E}C_1}, \quad t \rightarrow \infty. \quad (8.8.1)$$

If additionally $\mathbb{E}C_1^2 < \infty$ and $\text{Var}Z_1 < \infty$, then:

$$\sqrt{t}(\hat{X}(t) - I) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma_A^2),$$

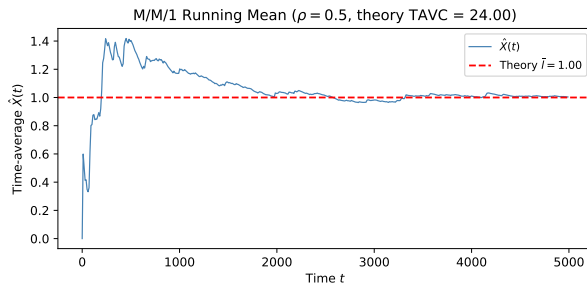
where $\varsigma_A^2 = \mathbb{E}\zeta_1^2 / \mathbb{E}C_1$, $\zeta_k = Z_k - I C_k$.

Regenerative Estimator

Using k complete cycles after T_1 :

$$\hat{I}_k = \frac{Z_1 + \cdots + Z_k}{C_1 + \cdots + C_k}, \quad \varsigma_B^2 = \frac{\mathbb{E}\zeta_1^2}{(\mathbb{E}C_1)^2}. \quad (8.8.6, 8.8.7)$$

On-Off Process: Estimator Comparison



Example (Table 8.2):

True $p = 3/4$. Two options:

- Opt. 1: $T^{\text{on}} \sim \text{Exp}(1/3)$, $T^{\text{off}} \sim \text{Exp}(1)$
- Opt. 2: $T^{\text{on}} \sim \text{Par}(2.5, 1.8)$,
 $T^{\text{off}} \sim \text{Gamma}(2, 1/2)$

	Opt. 1	Opt. 2
$\hat{X}(t_{\max})$	0.727	0.739
$\hat{I}_k$	0.725	0.739

Both converge to true $p = 0.75$. $\hat{I}_k$ is unbiased (discards first cycle).

Python: Regenerative Estimation

```
import numpy as np

def regenerative_onoff(lam_on, lam_off, n_cycles, rng=None):
    """Regenerative estimator for on-off process (Exp distributions)."""
    if rng is None:
        rng = np.random.default_rng()
    Zs, Cs = [], []
    for _ in range(n_cycles):
        T_on = rng.exponential(1/lam_on)
        T_off = rng.exponential(1/lam_off)
        C = T_on + T_off
        Z = T_on          # integral of state=1 over cycle
        Zs.append(Z); Cs.append(C)
    Zs, Cs = np.array(Zs), np.array(Cs)
    # Discard first cycle (non-delayed start)
    Z1, C1 = Zs[1:], Cs[1:]
    I_hat = np.sum(Z1) / np.sum(C1)    # regenerative estimator
    # TAVC estimate
    I_cycle = np.sum(Z1) / np.sum(C1)
    zeta = Z1 - I_cycle * C1
    sigma2_B = np.var(zeta, ddof=1) / (np.mean(C1)**2)
    return I_hat, sigma2_B

rng = np.random.default_rng(3)
I_hat, s2 = regenerative_onoff(1/3, 1.0, 300, rng)
print(f"I_hat = {I_hat:.4f}, TAVC = {s2:.4f}")
print(f"95% CI: ({I_hat - 1.96*np.sqrt(s2/300):.4f}, {I_hat + 1.96*np.sqrt(s2/300):.4f})")
```


8.9 Error Analysis of M/M/1 Queue

Steady-State Mean (8.9.2)

$$\bar{l} = \mathbb{E}[L^*(0)] = \frac{\rho}{1 - \rho}, \quad \rho = \lambda/\mu < 1.$$

Asymptotic variance (8.9.3):

$$\varsigma_L^2 = \frac{2\rho(1 + \rho)}{\mu(1 - \rho)^4}.$$

Required Simulation Length

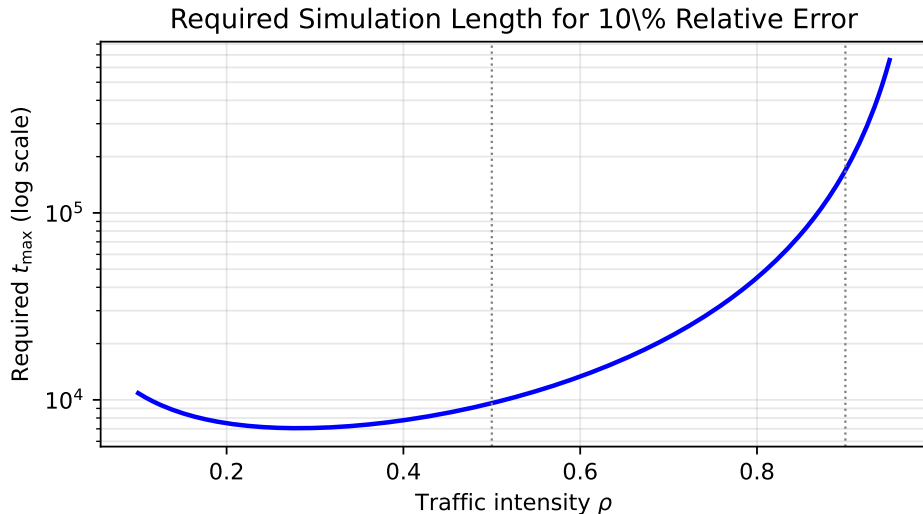
SACV = ς^2/l^2 . For $b = 0.1l$ (10% relative error), $\alpha = 0.05$:

$$t_{\max} = 400 \cdot \text{SACV}, \quad \text{SACV} = \frac{2(1 + \rho)}{\mu\rho(1 - \rho)^2}.$$

Numerical example:

- $\rho = 1/2$: SACV = 24, $t_{\max} = 9600$. Sim. gave $\hat{l} = 0.9822$ vs $\bar{l} = 1$.
- $\rho = 8/9$: SACV = 344.25, $t_{\max} = 1.377 \times 10^5$. Sim. gave $\hat{l} = 6.67$ vs $\bar{l} = 8$.

M/M/1 TAVC and Run Length vs. ρ



- Required run length grows as $(1 - \rho)^{-2}$ for *relative* error.

8.9 Error Analysis of M/G/ ∞ Systems

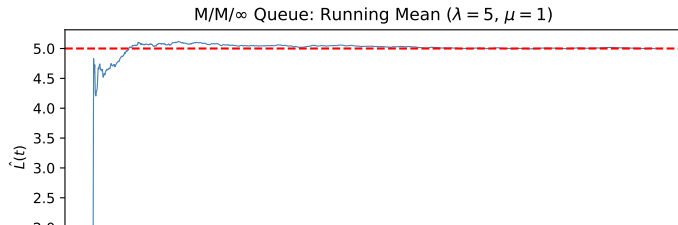
M/G/ ∞ Covariance and TAVC (8.9.4)

$$R(t) = \lambda \int_t^\infty (1 - G(v)) dv, \quad \zeta^2 = 2 \int_0^\infty R(t) dt = 2\lambda \int_0^\infty \int_t^\infty (1 - G(v)) dv dt.$$

Special case $G = \text{Exp}(\mu)$: $\zeta^2 = 2\lambda/\mu = 2\rho$.

Sensitivity to service distribution:

- Light-tailed G (Exp): $\zeta^2 < \infty$, fast convergence.
- Heavy-tailed $G = \text{Par}(\alpha)$:
 - $\alpha > 2$: finite variance, $\zeta^2 < \infty$.
 - $1 < \alpha \leq 2$: infinite variance, $\zeta^2 = \infty$; CLT may fail.



Python: M/G/ ∞ Simulation

```
import numpy as np

def simulate_mginf(lam, G_sample, t_max, rng=None):
    """Simulate M/G/inf queue and estimate steady-state mean."""
    if rng is None:
        rng = np.random.default_rng()
    # Generate arrivals (Poisson process)
    arrivals = []
    t = rng.exponential(1/lam)
    while t < t_max:
        arrivals.append(t); t += rng.exponential(1/lam)
    arrivals = np.array(arrivals)
    service = G_sample(len(arrivals), rng)
    departures = arrivals + service
    # Estimate: l_hat = integral of L(t) / t_max
    # Using formula (8.9.5): sum of min(t_max, D_i) - A_i
    L_integral = np.sum(np.minimum(departures, t_max) - arrivals)
    l_hat = L_integral / t_max
    return l_hat

rng = np.random.default_rng(99)
# Exponential service: E[S]=1, rho=5
l_hat = simulate_mginf(5.0, lambda n, r: r.exponential(1.0, n), 1e4, rng)
print(f"M/M/inf estimate: {l_hat:.4f}, theory rho=5.0")
# Pareto service (heavy-tailed)
l_hat2 = simulate_mginf(5.0, lambda n, r: 2.2*(r.pareto(3.2, n)+1), 1e4, rng)
print(f"M/Par/inf estimate: {l_hat2:.4f}")
```

8.10 Bibliographical Notes

Key references:

- **Poisson processes:** Billingsley [14] for theoretical foundations.
- **Queueing theory:** Ross [175–177]; Asmussen [7]; Kleinrock [117,118].
- **TAVC / SACV:** Kopocinski [111]; Whitt [213] (B&D formula); Reynolds [173].
- **Regenerative processes:** Asmussen & Glynn [8]; Thorisson [199].
- **Discrete-event simulation:** Law [130]; Bratley et al. [22].
- **Random access protocols:** Bertsekas & Gallager [13] (ALOHA).

Summary of Chapter 8 algorithms:

- Alg. 73: $\text{PoiPro}(\lambda)$ – homogeneous Poisson
- Alg. 74: $\text{PoiProSorted}(\lambda)$ – order-statistics method
- Alg. 75: $\text{NH-Poi-thinning}(\lambda(t))$ – NHPP via thinning
- Alg. 76: $\text{NH-Poi}(\lambda(t))$ – NHPP via inverse CDF
- Alg. 77: Poisson process on E – general spaces
- Alg. 78: BD – birth-and-death process
- Alg. 79: Repairman problem
- Alg. 80–81: Discrete-event simulation of GI/G/c queue
- Alg. 82: M/G/ ∞ simulation

Chapter 8 Summary

Arrival modeling:

- Homogeneous Poisson: $\tau_i \sim \text{Exp}(\lambda)$
- Non-homogeneous Poisson: thinning
- General spaces: Algorithm 77

Queue models:

- M/M/1, M/M/c: B&D processes
- GI/G/1: Lindley recursion
- GI/G/c: workload vector
- M/G/ ∞ : infinite servers

Output analysis:

- Time average $\hat{X}(t)$ vs. ensemble $\mathbb{E}[X^*]$
- TAVC: $\varsigma^2 = 2 \int_0^\infty R(v) dv$
- Confidence intervals: $\hat{X}(t) \pm z_{1-\alpha/2} \varsigma / \sqrt{t}$
- Regenerative estimator $\hat{I}_k$

Key formulas:

- M/M/1: $\bar{I} = \rho / (1 - \rho)$
- Little: $\bar{I} = \lambda \bar{w}$
- M/M/1 TAVC: $\varsigma^2 = 2\rho(1 + \rho) / [\mu(1 - \rho)^4]$